

# **CLASSIFICATION TECHNIQUES**

# DECISION TREE

---

# Decision Tree:

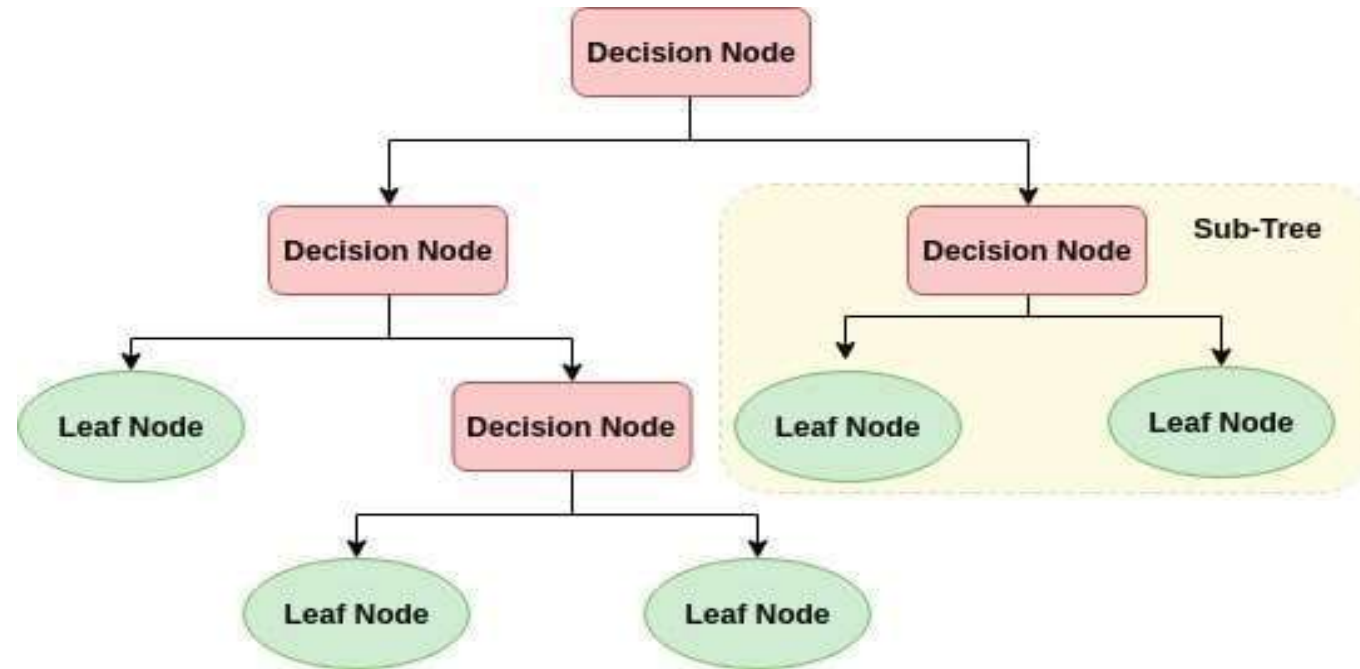
- As a loan manager, you need to identify risky loan applications to achieve a lower loan default rate. This process of classifying customers into a group of potential and non-potential customers or safe or risky loan applications is known as a classification problem.
- Classification is a two-step process, learning step and prediction step.
- In the learning step, the model is developed based on given training data.
- In the prediction step, the model is used to predict the response for given data.
- Decision Tree is one of the easiest and popular classification algorithms to understand and interpret. It can be utilized for both classification and regression kind of problem.

## CONTENTS:

- Decision Tree Algorithm
- How does the Decision Tree algorithm work?
- Attribute Selection Measures
  - Information Gain
  - Gain Ratio
  - Gini index
- Optimizing Decision Tree Performance
- Classifier Building in Scikit-learn
- Pros and Cons
- Conclusion

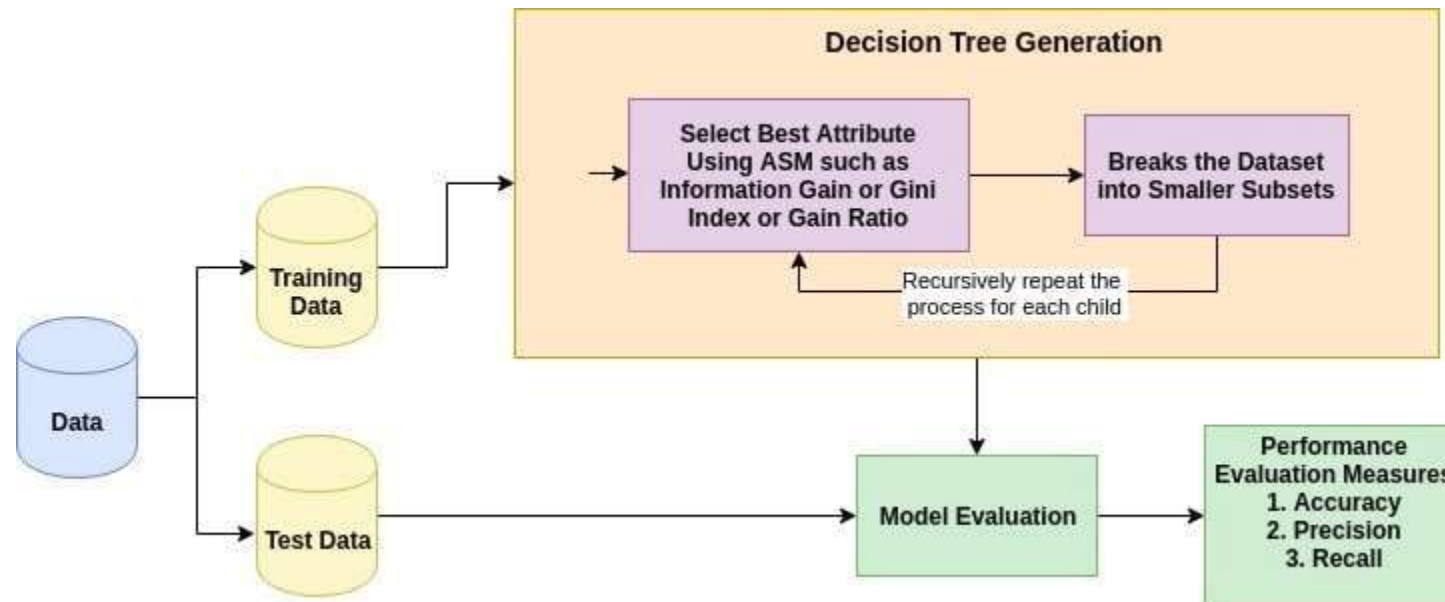
# Decision Tree Algorithm:

- A decision tree is a flowchart-like tree structure where an internal node represents feature(or attribute), the branch represents a decision rule, and each leaf node represents the outcome.
- The topmost node in a decision tree is known as the root node.
- It learns to partition on the basis of the attribute value.
- It partitions the tree in recursively manner call recursive partitioning.
- This flowchart-like structure helps you in decision making.
- It's visualization like a flowchart diagram which easily mimics the human level thinking. That is why decision trees are easy to understand and interpret.



# How Does The Decision Tree Algorithm Work:

- The basic idea behind any decision tree algorithm is as follows:
- Select the best attribute using Attribute Selection Measures(ASM) to split the records.
- Make that attribute a decision node and breaks the dataset into smaller subsets.
- Starts tree building by repeating this process recursively for each child until one of the condition will match:
  - All the tuples belong to the same attribute value.
  - There are no more remaining attributes.
  - There are no more instances.

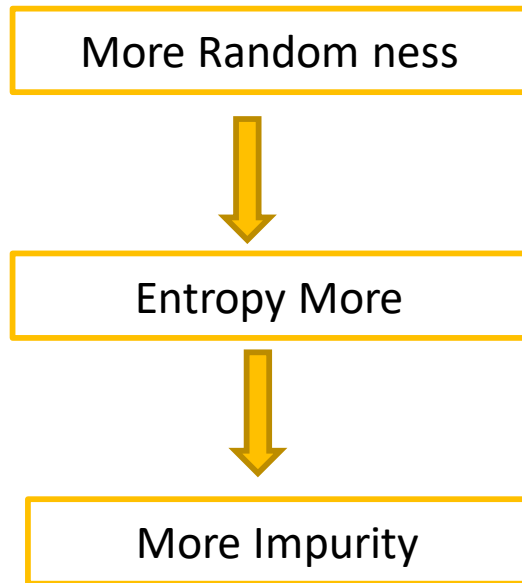


## **Attribute Selection Measure:**

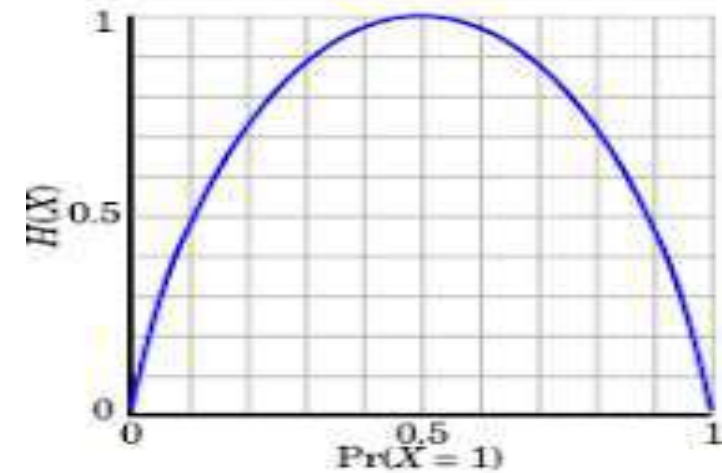
- Attribute selection measure is a heuristic for selecting the splitting criterion that partition data into the best possible manner.
- It is also known as splitting rules because it helps us to determine breakpoints for tuples on a given node.
- ASM provides a rank to each feature(or attribute) by explaining the given dataset.
- Best score attribute will be selected as a splitting attribute. Most popular selection measures are Information Gain, and Gini Index

## **Information Gain:**

- Shannon invented the concept of entropy, which measures the impurity of the input set.
- In physics and mathematics, entropy referred as the randomness or the impurity in the system.
- In information theory, it refers to the impurity in a group of examples.
- Information gain is the decrease in entropy.
- Information gain computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values.
- ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.



Shannon Entropy



$$\text{Info}(D) = - \sum_{i=1}^m p_i \log_2 p_i \quad \text{Info}_A(D) = \sum_{j=1}^V \frac{|D_j|}{|D|} \times \text{Info}(D_j) \quad \text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

Where,

- $p_i$  is the probability that an arbitrary tuple in  $D$  belongs to class  $C_i$ .
- $\text{Info}(D)$  is the average amount of information needed to identify the class label of a tuple in  $D$ .
- $|D_j|/|D|$  acts as the weight of the  $j$ th partition.
- $\text{Info}_A(D)$  is the expected information required to classify a tuple from  $D$  based on the partitioning by  $A$ .

Example: Table1

| Id | age         | income | student | Credit_rating | Class:<br>Buys computer |
|----|-------------|--------|---------|---------------|-------------------------|
| 1  | Youth       | High   | No      | Fair          | No                      |
| 2  | Youth       | High   | No      | Excellent     | No                      |
| 3  | Middle_aged | High   | No      | Fair          | Yes                     |
| 4  | Senior      | Medium | No      | Fair          | Yes                     |
| 5  | Senior      | Low    | Yes     | Fair          | Yes                     |
| 6  | Senior      | Low    | Yes     | Excellent     | No                      |
| 7  | Middle_aged | Low    | Yes     | Excellent     | Yes                     |
| 8  | Youth       | Medium | No      | Fair          | No                      |
| 9  | Youth       | Low    | Yes     | Fair          | Yes                     |
| 10 | Senior      | Medium | Yes     | Fair          | Yes                     |
| 11 | Youth       | Medium | Yes     | Excellent     | Yes                     |
| 12 | Middle_aged | Medium | No      | Excellent     | Yes                     |
| 13 | Middle_aged | High   | Yes     | Fair          | Yes                     |
| 14 | senior      | Medium | no      | Excellent     | No                      |



- The class label attribute, *buys computer*, has two distinct values (namely, *yes*, *no*);
- therefore, there are two distinct
- classes (i.e.,  $m \geq 2$ ). Let class *C1* correspond to *yes* and class *C2* correspond to *no*.
- There are nine tuples of class *yes* and five tuples of class *no*. A (root) node *N* is created for the tuples in *D*.
- To find the splitting criterion for these tuples, we must compute the information gain of each attribute.
- The total output attribute information gain is:

$$\text{Info}(D) = (-9/14) \log_2 \left( \frac{9}{14} \right) - \left( \frac{5}{14} \right) \log_2 \left( \frac{5}{14} \right) = 0.940$$

- Next, we need to compute the expected information requirement for each attribute.
- Let's start with the attribute *age*. We need to look at the distribution of *yes* and *no* tuples
- for each category of *age*. For the *age* category "youth," there are two *yes* tuples and three *no* tuples. For the category "middle aged," there are four *yes* tuples and zero *no* tuples.
- For the category "senior," there are three *yes* tuples and two *no* tuples

$$\begin{aligned} \text{info}(D)_{\text{age}} = & \left( \frac{5}{14} \right) \left( -\frac{2}{5} \log_2 \left( \frac{2}{5} \right) - \frac{3}{5} \log_2 \left( \frac{3}{5} \right) \right) + \left( \frac{4}{14} \right) \left( -\frac{4}{4} \log_2 \left( \frac{4}{4} \right) \right) \\ & + \left( \frac{5}{14} \right) \left( -\frac{3}{5} \log_2 \left( \frac{3}{5} \right) - \frac{2}{5} \log_2 \left( \frac{2}{5} \right) \right) = 0.694 \end{aligned}$$

Therefore

$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}(D)_{\text{age}} = 0.940 - 0.694 = 0.246 \text{ bits}$$

- Similarly, we can compute  $\text{Gain}(\text{income}) = 0.029$  bits,
- $\text{Gain}(\text{student}) = 0.151$  bits,
- and  $\text{Gain}(\text{credit rating}) = 0.048$  bits.
- Because *age* has the highest information gain
- among the attributes, it is selected as the splitting attribute. Node *N* is labeled with *age*,
- and branches are grown for each of the attribute's values.

### Gain Ratio:

- Information gain is biased for the attribute with many outcomes.
- It means it prefers the attribute with a large number of distinct values.
- For instance, consider an attribute with a unique identifier such as ID has zero  $\text{info}(D)$  because of pure partition.
- This maximizes the information gain and creates useless partitioning.
- C4.5, an improvement of ID3, uses an extension to information gain known as the gain ratio.
- Gain ratio handles the issue of bias by normalizing the information gain using Split Info.

$$\text{SplitInfo}_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left( \frac{|D_j|}{|D|} \right) \quad \longrightarrow \quad \text{GainRatio}(A) = \frac{\text{Gain}(A)}{\text{SplitInfo}_A(D)}$$

## Gini Index:

Another decision tree algorithm CART (Classification and Regression Tree) uses the Gini method to create split points.

$$\text{Gini}(D) = 1 - \sum_{i=1}^m p_i^2$$

- Where,  $p_i$  is the probability that a tuple in  $D$  belongs to class  $C_i$ .
- The Gini Index considers a binary split for each attribute.
- You can compute a weighted sum of the impurity of each partition.
- If a binary split on attribute  $A$  partitions data  $D$  into  $D_1$  and  $D_2$ , the Gini index of  $D$  is:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

The attribute with minimum Gini index is chosen as the splitting attribute.

$$\Delta \text{Gini}(A) = \text{Gini}(D) - \text{Gini}_A(D).$$

- Let  $D$  be the training data shown
- Earlier in Table , where there are nine tuples belonging to the class *buys computer Yes* and the remaining five tuples belong to the class *buys computer No*
- $N$  is created for the tuples in  $D$ .
- The Gini index to compute the impurity of  $D$ :

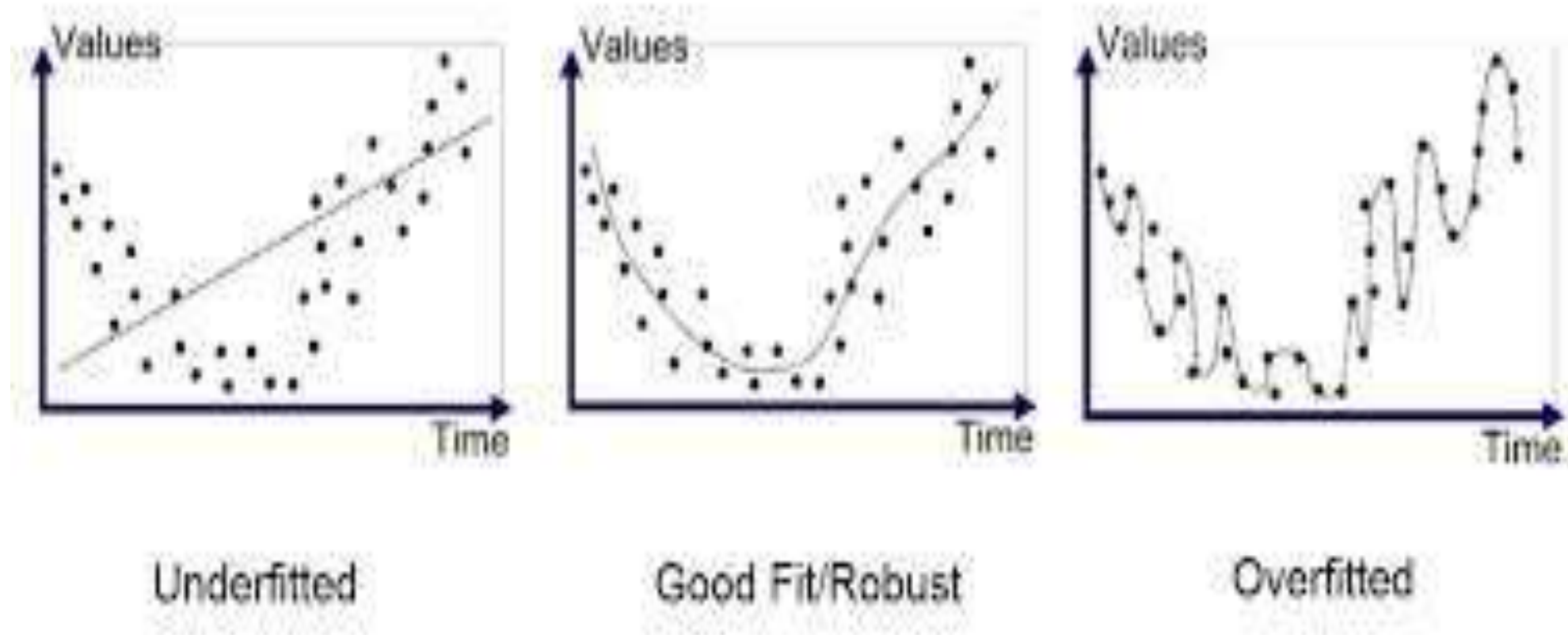
$$\text{Gini}(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

$$\begin{aligned} \text{Gini}(\text{Income}) \{ \text{low}, \text{medium} \} &= \left(\frac{10}{14}\right) \text{Gini}(D1) + \left(\frac{4}{14}\right) \text{Gini}(D2) \\ &= \left(\frac{10}{14}\right) \left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \left(\frac{4}{14}\right)^2 \left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) \\ &= 0.443 = \text{Gini}(\text{income}) \{ \text{high} \} \end{aligned}$$

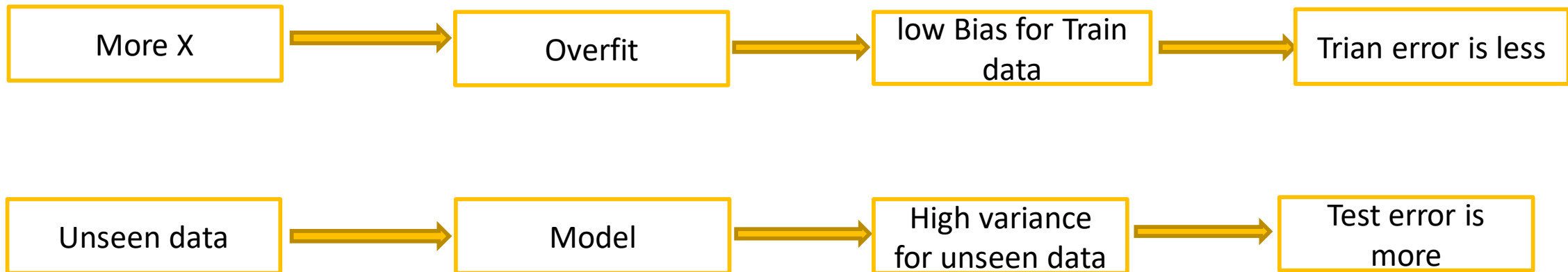
- Similarly, the Gini index values for splits on the remaining subsets are 0.458 {low,high} and {medium}
- and 0.450 {medium, high} and {low}.
- Therefore, the best binary split for attribute *income* is on {low,medium} and {high} is high
- because it minimizes the Gini index

## Decision Tree-Over fitting(Tree Pruning):

Machine learning is a problem of trade-offs. The classic issue is overfitting versus underfitting. Overfitting happens when a model memorizes its training data so well that it is learning noise on top of the signal. Underfitting is the opposite: the model is too simple to find the patterns in the data.



- If you observe the diagram suppose x-axis is =Size, y-axis is =Values
- During train time if you increases x(size) value increases our model fits the data very well, that means the accuracy will increase, by adding variables
- In the diagram
  - The first picture has less Attributes(X) it leads to underfit
  - The second picture has slightly more Attributes( X) it leads to Good fit
  - The Third Picture has a lot Attributes (X) it leads to Overfit
- As X=====➔Increases
- Our model **becomes Brainless** for Training data , It nothing but **just Byhearting outputs**
- When you given a test data or unseen data to this model it **leads High variance**
- That means it get more error for test data



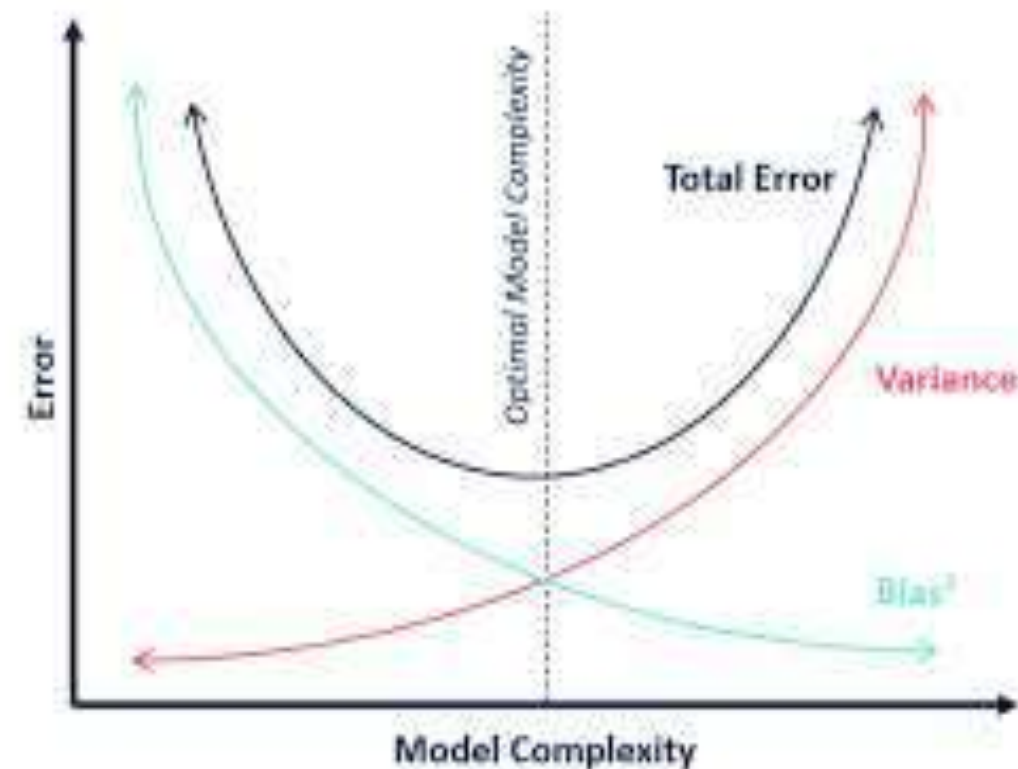
## Bias –Variance Trade Off:

### Key points:

- Y-axis= Mean error
- X-axis = Model complexity
- As model increases the bias will decrease and variance will increase For unseen data(test) data
- When bias is more variance is less
- **High Bias= Low variance**
- **Low Bias= High Variance**

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{var}(\text{error})$$

The above formulae should reduce to get model



As the model complexity increases ,training error decreases corresponding test error is decreases.

The CART algorithm will repeatedly partition data into smaller and smaller subsets until those final subsets are homogeneous in terms of the outcome variable. In practice this often means that the final subsets (known as the *leaves* of the tree) each consist of only one or a few data points. The tree has learned the data exactly, but a new data point that differs very slightly might not be predicted well.

## Two approaches to Picking up Simpler trees:

### Early stopping(Pre-pruning):

Stop the learning algorithm before the tree becomes too complex

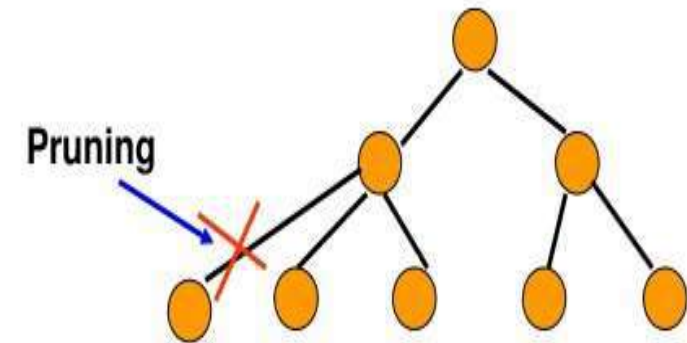
### Post-Pruning:

**Pruning** is a technique in machine learning and search algorithms that reduces the size of decision trees by removing sections of the tree that provide little power to classify instances. Pruning reduces the complexity of the final\_classifier, and hence improves predictive accuracy by the reduction of overfitting.

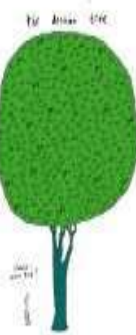
simplify the tree after the learning algorithm terminates

## How to Build Decision Tree?

- Generally, building a decision tree involved 2 steps:
- Tree construction** → recursively split the tree according to selected attributes (conditions),
- Tree pruning** → identify and remove the irrelevance branches (that might lead to outliers) – to increase classification accuracy.



By : Mohd. Noor Abdul Hamid, Ph.D  
(Universiti Utara Malaysia)





## **Steps To follow Tree Pruning:**

- We first make the decision tree to a large depth.
- Then we start at the bottom and start removing leaves which are giving us negative returns when compared from the top.
- Suppose a split is giving us a gain of say -10 (loss of 10) and then the next split on that gives us a gain of 20. A simple decision tree will stop at step 1 but in pruning, we will see that the overall gain is +10 and keep both leaves.

# Naive Bayes

---

## Naive Bayes':

- Bayesian classifiers are statistical classifiers.
- They can predict class membership probabilities such as the probability that a given tuple belongs to a Particular-class.
- Bayesian classification is based on **Bayes' theorem**.
- Studies comparing classification algorithms have found a simple Bayesian classifier known as **the *naive***
- *Bayesian classifier* to be comparable in performance with decision tree and selected neural network classifiers.
- Bayesian classifiers have also exhibited high accuracy and speed when applied to large databases.

## Assumption:

- Naive Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes.
- This assumption is called ***class conditional independence***.
- It is made to simplify the computations involved and, in this sense, is considered “naive.”

# Bayes' Theorem:

Bayes' theorem is named after Thomas Bayes

- Let  $\mathbf{X}$  be a data tuple.
- Let  $H$  be some hypothesis such as that the data tuple  $\mathbf{X}$  belongs to a specified class  $C$ .
- For classification problems, we want to determine  $P(H/X)$

## Some Definitions:

### Posterior Probability:

**$P(H/X)$  is the posterior probability, or a *posteriori* probability, of  $H$  conditioned on  $X$**

### Example:

- that  $\mathbf{X}$  is a 35-year-old customer with an income of \$40,000.
- Suppose that  $H$  is the hypothesis that our customer will buy a computer.
- Then  $P(H/X)$  reflects the probability that customer  $\mathbf{X}$  will buy a computer given that we know the customer's age and income.

### Prior Probability Of $H$ :

**$P(H)$  is the prior probability, or a *priori* probability, of  $H$ .**

- For our example, this is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter.
- The posterior probability,  $P(H/X)$  is based on more information (e.g., customer information) than the prior probability,
- $P(H)$  which is independent of  $\mathbf{X}$ .

## Posterior probability:

**$P(X/H)$  is the posterior probability of  $X$  conditioned on  $H$ .**

- That is, it is the probability that a customer,  $X$ , is 35 years old and earns \$40,000, given that we know the customer will buy a computer.

## Prior Probability Of $X$ :

**$P(X)$  is the prior probability of  $X$ .**

- Using our example, it is the probability that a person from our set of customers is 35 years old and earns \$40,000.

## *“How are these probabilities estimated?”*

$P(H)$ ,  $P(X/H)$ , and  $P(X)$  may be estimated from the given data

- **Bayes' theorem** is useful in that it provides a way of calculating the posterior probability,  $P(H/X)$ , from  $P(H)$ ,  $P(X/H)$ , and  $P(X)$ .

Bayes' theorem is

$$P(H/X) = \frac{P(X/H)P(H)}{P(X)}$$

## Naive Bayes' Classification:

- Let  $D$  be a training set of tuples and their associated class labels.
- As usual, each tuple is represented by an  $n$ -dimensional attribute vector,  $\mathbf{X}=(x_1,x_2,x_3,\dots,x_n)$
- Suppose that there are  $m$  classes,  $C_1, C_2, \dots, C_m$ . Given a tuple,  $\mathbf{X}$ , the classifier will predict that  $\mathbf{X}$  belongs to the class having the highest posterior probability, conditioned on  $\mathbf{X}$ .
- That is, the naive Bayesian classifier predicts that tuple  $\mathbf{X}$  belongs to the class  $C_i$
- if and only if

$$P(C_i/\mathbf{X}) > P(C_j/\mathbf{X}) \quad \text{for } 1 \leq j \leq m. \{i \text{ not equal to } j\}$$

- Thus, we maximize  $P(C_i/\mathbf{X})$ .
- The class  $C_i$  for which  $P(C_i/\mathbf{X})$  is maximized is called the **maximum posteriori hypothesis**.

$$P(C_i/\mathbf{X}) = \frac{P(\mathbf{X}/C_i)P(C_i)}{P(\mathbf{X})}$$

- $P(\mathbf{X})$  is same for all classes, only  $P(\mathbf{X}/C_i)P(C_i)$
- Given data sets with many attributes, it would be extremely computationally expensive to compute  $P(\mathbf{X}/C_i)$
- To reduce computation in evaluating , the  $P(\mathbf{X}/C_i)$ , naive assumption of **class-conditional independence** is made. This presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple

$$P(\mathbf{X}/C_i) = \prod_{k=1}^n P(X_k/C_i) = P(X_1/C_i) P(X_2/C_i) P(X_3/C_i) \dots$$

- We can easily estimate the probabilities  $P(X1/Ci)$ ,  $P(X2/Ci)$ , ..... ,  $P(Xn/Ci)$  from the training tuples.
- Recall that here  $Xk$  refers to the value of attribute  $Ak$  for tuple  $\mathbf{X}$ .
- For each attribute, we look at whether the attribute is categorical or continuous-valued.
- For instance, to compute  $P(\mathbf{X}/Ci)$ , we consider the following:
  - (a) If  $Ak$  is categorical, then  $P(Xk/Ci)$  is
    - the number of tuples of class  $Ci$  in  $D$  having the value  $Xk$  for  $Ak$ ,
    - divided by the number of tuples of class  $Ci$  in  $D$ .
  - (b) If  $Ak$  is continuous-valued,
    - Then we need to do a bit more work, but the calculation is pretty straightforward.
    - A continuous-valued attribute is typically assumed to have a Gaussian distribution with a mean and standard deviation

$$y = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x - \mu)^2}{2\sigma^2}}$$

$\mu =$  Mean

$\sigma =$  Standard Deviation

$\pi \approx 3.14159 \dots$

$e \approx 2.71828 \dots$

$$P(Xk/Cj)=y(Xk,Mean(Ci),Sd(Ci))$$

Look at The Previous example Table that we solved for Decision Tree, Now same problem we will do for Naive Bayes.

- The data tuples are described by the attributes *age*, *income*, *student*, and *credit rating*.
- The class label attribute, *buys computer*, has two distinct values namely(*yes*, *no*).
- Let  $C_1$  correspond to the class *buys computer* = *yes* and  $C_2$  correspond to *buys computer* = *no*.

### Question:

The tuple we wish to classify is

**$X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit\_rating} = \text{fair})$**

### **Step1:**

Calculate  $P(C_i)$  The prior probability of each class

- $P(\text{buys computer} = \text{yes}) = 9/14 = 0.643$
- $P(\text{buys computer} = \text{no}) = 5/14 = 0.357$

### **Step2:**

Compute  $P(X/C_i)$

- $P(\text{age} = \text{youth} \mid \text{buys computer} = \text{yes}) = 2/9 = 0.222$
- $P(\text{age} = \text{youth} \mid \text{buys computer} = \text{no}) = 3/5 = 0.600$
- $P(\text{income} = \text{medium} \mid \text{buys computer} = \text{yes}) = 4/9 = 0.444$
- $P(\text{income} = \text{medium} \mid \text{buys computer} = \text{no}) = 2/5 = 0.400$
- $P(\text{student} = \text{yes} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$



- $P(\text{student} = \text{yes} \mid \text{buys computer} = \text{no}) = 1/5 = 0.200$
- $P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{yes}) = 6/9 = 0.667$
- $P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{no}) = 2/5 = 0.400$

Using these probabilities, we obtain

- $P(\mathbf{X} \mid \text{buys computer} = \text{yes}) = P(\text{age} = \text{youth} \mid \text{buys computer} = \text{yes}) \times$   
 $P(\text{income} = \text{medium} \mid \text{buys computer} = \text{yes}) \times$   
 $P(\text{student} = \text{yes} \mid \text{buys computer} = \text{yes}) \times$   
 $P(\text{credit rating} = \text{fair} \mid \text{buys computer} = \text{yes})$   
 $= (0.222)(0.444)(0.667)(0.667) = 0.044.$
- Similarly,
- $P(\mathbf{X} \mid \text{buys computer} = \text{no}) = (0.600)(0.400)(0.200)(0.400) = 0.019.$

**Step3:**

**To find the class,  $C_i$ , that maximizes  $P(\mathbf{X} \mid C_i)P(C_i)$ ,**

we compute

- $P(\mathbf{X} \mid \text{buys computer} = \text{yes})P(\text{buys computer} = \text{yes}) = (0.044)(0.643) = 0.028$
- $P(\mathbf{X} \mid \text{buys computer} = \text{no})P(\text{buys computer} = \text{no}) = (0.019)(0.357) = 0.007$

**Therefore, the naive Bayesian classifier predicts *buys computer* D *yes* for tuple  $\mathbf{X}$ .**

## Laplacian Correction:

“What if I encounter probability values of zero?”

- Suppose that for the class *buys computer = yes*
- In some training database,  $D$ , containing 1000 tuples,
  - we have 0 tuples with *income = low*,
  - 990 tuples with *income = medium*, and
  - 10 tuples with *income = high*.
- The probabilities of these events, without the Laplacian correction, are
- 0, 0.990 (from 990/1000), and 0.010 (from 10/1000), respectively

If we apply the above values into before problem  $P(X/\text{buys computer} = \text{yes})$  it becomes Zero

**Laplacian correction is used to avoid computing probability values of zero.**

**Correction:**

- Laplacian correction for the three quantities, we pretend that we have 1 more tuple
- for each income-value pair. In this way, we instead obtain the following probabilities (rounded up to three decimal places):
- $1/1003 = 0.001$ ,  $991/1003 = 0.988$ , and  $11/1003 = 0.011$ , respectively.
- The “corrected” probability estimates are close to their “uncorrected” counterparts, yet the zero probability value is avoided.

### Questions For You:

- 1) Find out the difference between Decision Tree and Naive Bayes?
- 2) What are the Hyperparameters of both algorithms?

# **Model Evaluation Metrics**

---

# Model Evaluation and Selection:

- Now that you may have built a classification model, there may be many questions going through your mind.
- For example, suppose you used data from previous sales to build A classifier to predict customer purchasing behavior.
- You would like an estimate of how accurately the classifier can predict the purchasing behavior of future customers,
- That is, future customer data on which the classifier has not been trained.
- You may even have tried different methods to build more than one classifier and now wish to compare their accuracy.
- But what is accuracy? How can we estimate it?
- Are some measures of a classifier's accuracy more appropriate than others?
- How can we obtain a *reliable* accuracy estimate? These questions are addressed in this section.

# Confusion Matrix:

|        |     | Predicted |    |                   |
|--------|-----|-----------|----|-------------------|
|        |     | Yes       | No |                   |
| Actual | Yes | TP        | FN | P                 |
|        | No  | FP        | TN | N                 |
| Total  |     | P'        | N' | =(P+N) or (P'+N') |

**True positives (TP):**

These refer to the positive tuples that were correctly labeled by the classifier. Let *TP* be the number of true positives.

**True negatives (TN):**

These are the negative tuples that were correctly labeled by the classifier. Let *TN* be the number of true negatives.

**False positives (FP):**

These are the negative tuples that were incorrectly labeled as positive (e.g., tuples of class *buys computer =no* for which the classifier predicted *buys computer = yes*). Let *FP* be the number of false positives.

**False negatives (FN):**

These are the positive tuples that were mislabeled as negative (e.g., tuples of class *buys computer =yes* for which the classifier predicted *buys computer = no*). Let *FN* be the number of false negatives.

## Accuracy:

The **accuracy** of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier. That is

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$$

- This is also referred to as the overall **recognition rate** of the classifier

## Error Rate:

$$\text{error rate or misclassification rate} = (1 - \text{accuracy}) = \frac{\text{FP} + \text{FN}}{\text{P} + \text{N}}$$

## Class Imbalance Problem:

- Suppose you are working on a problem to detect the cancer patient, you labeled that
- if the patient has cancer =1, if the patient has not cancer=0
- that means your target variable is 1, generally in public rate of the disease is very low. So you will get more zeros(0) and less ones(1), this type of data is called data or class imbalance problem.

## What is the problem:

- suppose you have data set of 100 observations in that
- one person has cancer i.e 1, rest of all 99 persons has no cancer i.e 0
- Now you are building a model ,If your model is good to identified who are has not cancer, but unable to identified who actually has cancer. Suppose out of 100 members 99 are not has cancer and 1 person has cancer.
- if you apply your model it will find all 99 members has not cancer correctly, but it will not identified one person who actually gas cancer.
- **Then what is accuracy?**
- Yes....99% because your model identified 99 members correctly out off 100
- **But is this worth?**
- NO.....Because your model is missing target, here your target is identify cancer patient that are low actually .
- So accuracy fails in imbalance problem
- The **sensitivity** and **specificity** measures can be used, respectively, for this purpose



**Sensitivity:**

- Sensitivity is also referred to as the *true positive (recognition) rate*
- (i.e., the proportion of positive tuples that are correctly identified)

**Sensitivity=(TPR)=TP/P**

**Specificity:**

- specificity is the *true negative rate*
- (i.e., the proportion of negative tuples that are correctly identified).

**Specificity=(TNR)=TN/N**

**Accuracy=(sensitivity) $\frac{P}{P+N}$  + *Specificity*  $\frac{N}{P+N}$**

**Example:**

|        |       | Predicted |     |      |        |             |
|--------|-------|-----------|-----|------|--------|-------------|
|        |       | Classes   | Yes | No   | Total  | Recognition |
| Actual | Yes   |           | 90  | 210  | 300    | 30.00%      |
|        | No    |           | 140 | 9560 | 9700   | 98.56%      |
|        | Total |           | 230 | 9770 | 10,000 | 96.40       |

- The sensitivity of the classifier is  $\frac{90}{300} = 30.00\%$
- The specificity of the classifier is  $\frac{9560}{9700} = 98.56\%$
- The classifier Accuracy is  $\frac{9650}{10,000} = 96.50\%$
- Thus, we note that although the classifier has a high accuracy,
- It's ability to correctly label the positive (rare) class is poor given its low sensitivity.
- It has high specificity, meaning that it can accurately recognize negative tuples.

### Precision:

- **Precision** can be thought of as a measure of *exactness*.
- *Your model predicted some positive tuples, precision measures what is the percentage of match between actual and model positives(TP)*

$$\text{Precision} = \frac{TP}{TP + FP}$$

### Recall:

- **recall** is a measure of *completeness* (what percentage of positive tuples are labeled as such).
- If recall seems familiar, that's because it is the same as sensitivity (or the *true positive rate*).

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{TP}{P}$$

From the figure of example

The Precision is  $= \frac{90}{230} = 39.13\%$ ,

The Recall is  $= \frac{90}{300} = 30.00\%$

Questions For YOU:

- 1) What is meant by if Precision=1 and 0
- 2) What is meant by if Recall =1 and 0

- A perfect precision score of 1.0 for a class  $C$  means that every tuple that the classifier labeled as belonging to class  $C$  does indeed belong to class  $C$ .
- However, it does not tell us anything about the number of class  $C$  tuples that the classifier mislabeled.
- A perfect recall score of 1.0 for  $C$  means that every item from class  $C$  was labeled as such,
- but it does not tell us how many other tuples were incorrectly labeled as belonging to class  $C$ .
- There tends to be an inverse relationship between precision and recall, where it is possible to increase one at the cost of reducing the other.

### **F1score:**

Simply F1-score is harmonic average of precision and recall

### **why:?**

- Suppose classifier A has precision(p)=95%, Recall(r)=90%
- Suppose classifier A has precision(p)=98%, Recall(r)=85%
- if you compare precision and recall individually on both classifiers....
- Classifier A is good on Recall, Classifier B is good on Precision so which classifier you will choose....?
- **Confusion occurs:**
- That's why we go for F1 score its like Harmonic mean of two numbers

$$F_1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

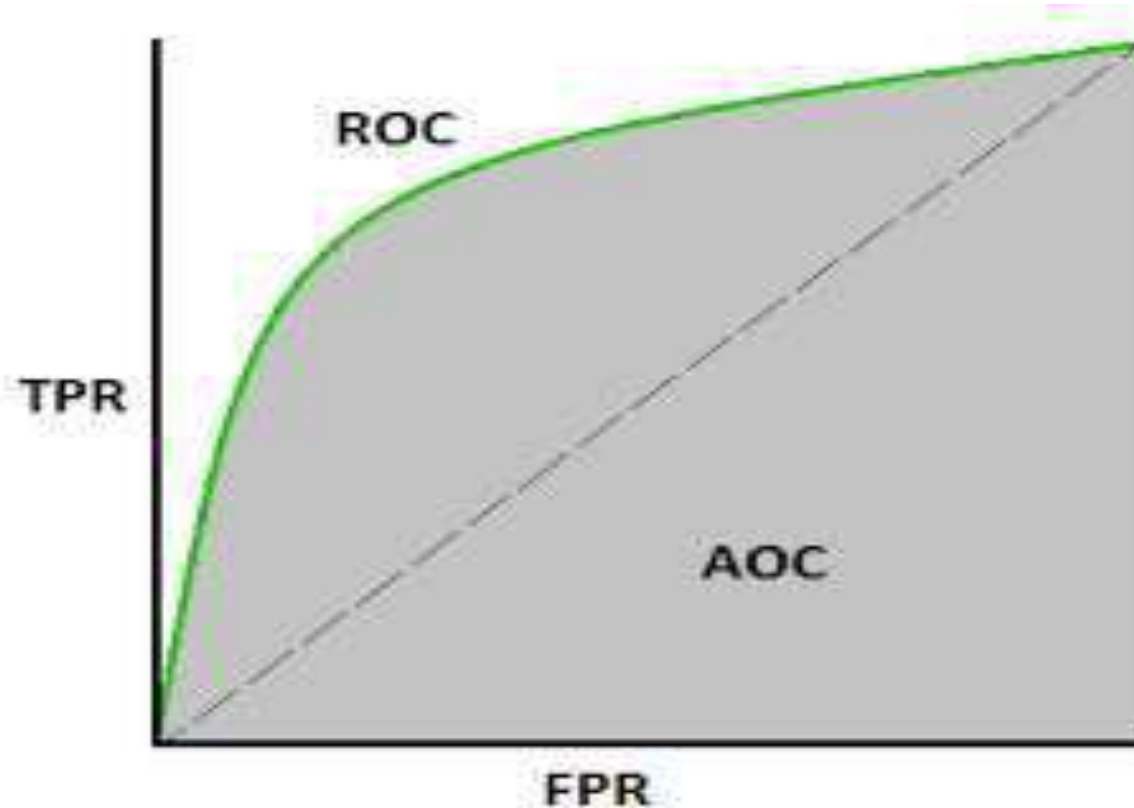
## **KEY POINTS:**

- 1) Do not confuse about confusion matrix
- 2) Follow your own method to remember confusion matrix
- 3) Then derive TP,TN,FP,FP
- 4) Sensitivity deals about only True Positives(TP)
- 5) Specificity deals with only True Negative(TN)
- 6) Precision and recall both are deal with True Positive(TP)
- 7) See numerator for precision and recall only denominator will change
- 8) That's why there is a trade off between them
- 9) F1 score give trade of both

# Receiver Operative Characteristics(ROC)

- **Receiver operating characteristic curves** are a useful visual tool for comparing two classification models.
- ROC curves come from signal detection theory that was developed during World War II for the analysis of radar images.
- An ROC curve for a given model shows the trade-off between the *true positive rate (TPR)* and the *false positive rate(FPR)*.
- Given a test set and a model, *TPR* is the proportion of positive (or “yes”) tuples that are correctly labeled by the model;
- *FPR* is the proportion of negative (or “no”) tuples that are mislabeled as positive.
- Given that *TP*, *FP*, *P*, and *N* are the number of true positive, false positive, positive, and negative tuples, respectively,
- we know that  $TPR = \frac{TP}{P}$  which is sensitivity.
- Furthermore,  $FPR = \frac{FP}{N}$  , which is (*1-specificity*.)

- For a two-class problem, An ROC curve allows us to visualize the trade-off between the rate at which the model can accurately recognize positive cases versus the rate at which it mistakenly identifies negative cases as positive for different portions of the test set.
- ROC curve is a measure of the accuracy of the model.



Example:

| Tuple | CLASS | Prob | TP | FP | TN | FN | TPR | FPR |
|-------|-------|------|----|----|----|----|-----|-----|
| 1     | P     | 0.90 | 1  | 0  | 5  | 4  | 0.2 | 0   |
| 2     | P     | 0.80 | 2  | 0  | 5  | 3  | 0.4 | 0   |
| 3     | N     | 0.70 | 2  | 1  | 4  | 3  | 0.4 | 0.2 |
| 4     | P     | 0.60 | 3  | 1  | 4  | 2  | 0.6 | 0.2 |
| 5     | P     | 0.55 | 4  | 1  | 4  | 1  | 0.8 | 0.2 |
| 6     | N     | 0.54 | 4  | 2  | 3  | 1  | 0.8 | 0.4 |
| 7     | N     | 0.53 | 4  | 3  | 2  | 1  | 0.8 | 0.6 |
| 8     | N     | 0.51 | 4  | 4  | 1  | 1  | 0.8 | 0.8 |
| 9     | P     | 0.50 | 5  | 4  | 1  | 0  | 1.0 | 0.8 |
| 10    | N     | 0.40 | 5  | 5  | 0  | 0  | 1.0 | 1.0 |



- In Figure shows the probability value (column 3) returned by a probabilistic classifier for each of the 10 tuples in a test set, sorted by decreasing probability order.
- Column 1 is merely a tuple identification number
- Column 2 is the actual class label of the tuple. There are five positive tuples and five negative tuples, thus  $P = 5$  and  $N = 5$ .
- As we examine the known class label of each tuple, we can determine the values of the remaining columns,  $TP$ ,  $FP$ ,  $TN$ ,  $FN$ ,  $TPR$ , and  $FPR$ .
- We start with tuple 1, which has the highest probability score, and take that score as our threshold, that is,  $t = 0.9$ .
- Thus, the classifier considers tuple 1 to be positive, and all the other tuples are considered negative.
- Since the actual class label of tuple 1 is positive, we have a true positive, hence  $TP = 1$  and  $FP = 0$ .
- Among the remaining nine tuples, which are all classified as negative, five actually are negative (thus,  $TN = 5$ ). The remaining four are all actually positive, thus,  $FN = 4$ .

Therefore  $TPR = \frac{TP}{P} = \frac{1}{5} = 0.2$ , While  $FPR = 0$

- we have the point (0.2,0) for the ROC curve.

- Next, threshold  $t$  is set to 0.8, the probability value for tuple 2, so this tuple is now also considered positive,
- The actual class label of tuple 2 is positive, thus now  $TP = 2$ . The rest of the row can easily be computed, resulting in the point (.0.4,0).
- Next, we examine the class label of tuple 3 and let  $t$  be 0.7, the probability value returned by the classifier for that tuple.
- Thus, tuple 3 is considered positive, yet its actual label is negative, and so it is a false positive. Thus,  $TP$  stays the same and  $FP$  increments so that  $FP = 1$ .
- The rest of the values in the row can also be easily computed, yielding the point (0.4, 0.2)

**Cohen's Kappa:**

Cohen's kappa coefficient ( $\kappa$ ) is a statistic that is used to measure inter-rater reliability for qualitative (categorical) items.

**Example:**

Suppose that you were analyzing data related to a group of 50 people applying for a grant. Each grant proposal was read by two readers and each reader either said "Yes" or "No" to the proposal. Suppose the disagreement count data were as follows, where A and B are readers, data on the main diagonal of the matrix (a and d) count the number of agreements and off-diagonal data (b and c) count the number of disagreements:

|   |     | B   |    |
|---|-----|-----|----|
|   |     | Yes | NO |
| A | YES | a   | B  |
|   | NO  | c   | d  |

|   |     | B   |    |
|---|-----|-----|----|
|   |     | Yes | No |
| A | Yes | 20  | 5  |
|   | No  | 10  | 15 |

## Steps to calculate cohen's kappa:

$$K = (p_o - p_e) / (1 - p_e)$$

$p_o$  = is the relative observed agreement among raters identical to accuracy

$p_e$  is the hypothetical probability of chance agreement, using the observed data to calculate the probabilities of each observer randomly seeing each category.

$$p_o = (a+d)/(a+b+c+d)$$

$$P_{yes} = [(a+b)/(a+b+c+d)] [(a+c)/(a+b+c+d)]$$

$$P_{no} = [(c+d)/(a+b+c+d)] [(b+d)/(a+b+c+d)]$$

$$p_e = P_{yes} + P_{no}$$

*So from example:*

$$p_o = (20+15)/50 = 0.7$$

$$P_{yes} = (0.5)(0.6) = 0.3$$

$$P_{no} = (0.5)(0.4) = 0.2$$

$$p_e = 0.3 + 0.2 = 0.5$$

$$K = (0.7 - 0.5) / (1 - 0.5) = 0.4$$

- The Kappa statistic varies from 0 to 1, where.
- 0 = agreement equivalent to chance.
- 0.1 – 0.20 = slight agreement.
- 0.21 – 0.40 = fair agreement.
- 0.41 – 0.60 = moderate agreement.
- 0.61 – 0.80 = substantial agreement.
- 0.81 – 0.99 = near perfect agreement
- 1 = perfect agreement

# K-Nearest Neighbors

---

# KNN:

we will talk about another widely used machine learning classification technique called K-nearest neighbors (KNN) . Our focus will be primarily on how does the algorithm work and how does the input parameter affect the output/prediction.

The KNN algorithm assumes that similar things exist in close proximity. In other words, similar things are near to each other.

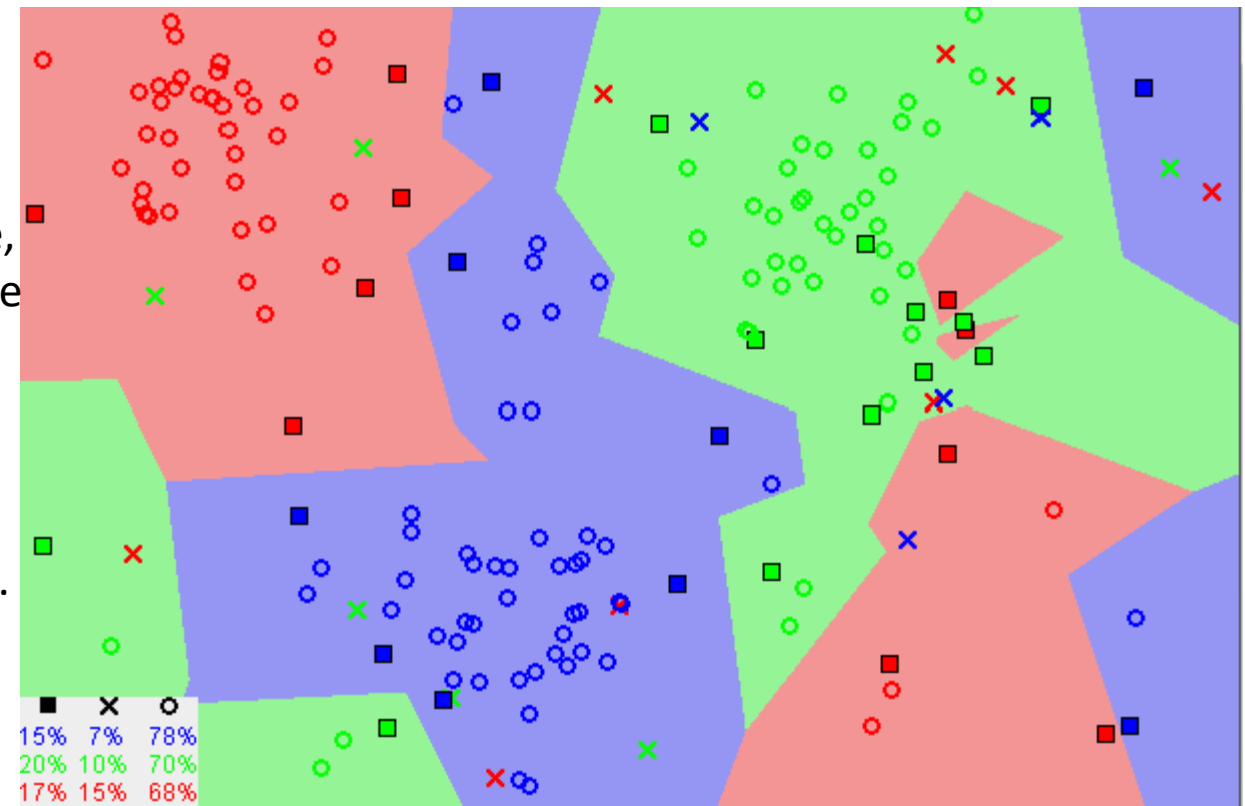
## **When do we use KNN algorithm?**

KNN can be used for both classification and regression predictive problems. However, it is more widely used in classification problems in the industry. To evaluate any technique we generally look at 3 important aspects:

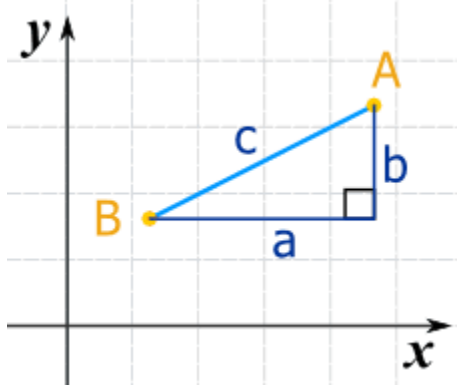
1. Ease to interpret output
2. Calculation time
3. Predictive Power

Notice in the image above that most of the time, similar data points are close to each other. KNN captures the idea of similarity (sometimes called distance, proximity, or closeness) with some mathematics we might have learned in our childhood— calculating the distance between points on a graph.

There are other ways of calculating distance, and one way might be preferable depending on the problem we are solving. However, the straight-line distance (also called the Euclidean distance) is a popular and familiar choice.



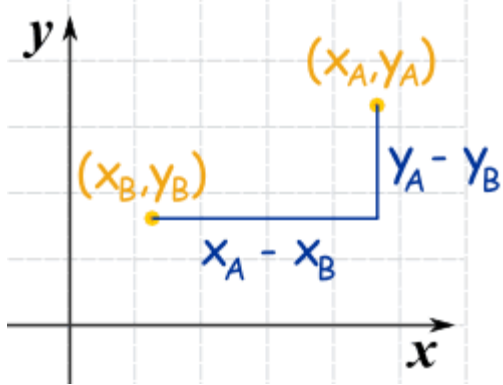




We can run lines down from **A**, and along from **B**, to make a Right-Angled Triangle.

And with a little help from Pythagoras we know that:

$$a^2 + b^2 = c^2$$



Now label the coordinates of points A and B.

$x_A$  means the x-coordinate of point A

$y_A$  means the y-coordinate of point A

The horizontal distance **a** is  $(x_A - x_B)$

The vertical distance **b** is  $(y_A - y_B)$

Start with:  $c^2 = a^2 + b^2$

Put in the calculations for a and b:  $c^2 = (x_A - x_B)^2 + (y_A - y_B)^2$

Square root of both sides:  $c = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$

# Steps To Follow KNN:

## The KNN Algorithm

- Load the data
- Initialize K to your chosen number of neighbors
- For each example in the data
  - Calculate the distance between the query example and the current example from the data.
  - Add the distance and the index of the example to an ordered collection
- Sort the ordered collection of distances and indices from smallest to largest (in ascending order) by the distances
- Pick the first K entries from the sorted collection
- Get the labels of the selected K entries
- If regression, return the mean of the K labels
- If classification, return the mode or take majority votes of the K labels

## Example:

Step 1:

Data:

We have from the people survey and objective testing with two attributes  $x_1$ =acid durability  
 $x_2$ =strength to classify whether a special paper tissue is good or not.

| X1 | X2 | Y    |
|----|----|------|
| 7  | 7  | Bad  |
| 7  | 4  | Bad  |
| 3  | 4  | Good |
| 1  | 4  | Good |

Question:

If new tuple  $x_1=3$ , and  $x_2=7$  what is Y?

**Step2:** Determine K value, assume k=3

**Step3:** Calculate distance between Query tuple with all training tuple

Query tuple (3,7)

| X1 | X2 | Distance(Euclidian)  |
|----|----|----------------------|
| 7  | 7  | $(7-3)^2+(7-7)^2=16$ |
| 7  | 4  | $(7-3)^2+(4-7)^2=25$ |
| 3  | 4  | $(3-3)^2+(4-7)^2=9$  |
| 1  | 4  | $(1-3)^2+(4-7)^2=13$ |

**Step4:** Sort the distance and determine nearest neighbors based on k-th minimum distance

| X1 | X2 | Distance(in square) | Rank | Is it including $k \leq 3$ |
|----|----|---------------------|------|----------------------------|
| 7  | 7  | 16                  | 3    | Yes                        |
| 7  | 4  | 25                  | 4    | No                         |
| 3  | 4  | 9                   | 1    | Yes                        |
| 1  | 4  | 13                  | 2    | Yes                        |

**Step5:** Gather the Y category of training data examples

| X1 | X2 | Distance | Rank | Is it includes<br>k<=3 | Y     |
|----|----|----------|------|------------------------|-------|
| 7  | 7  | 16       | 3    | Yes                    | Bad   |
| 7  | 4  | 25       | 4    | No                     | ----- |
| 3  | 4  | 9        | 1    | Yes                    | Good  |
| 1  | 4  | 13       | 2    | Yes                    | Good  |

Row 2, rank is 4  
That means it is not in nearest neighbors so we will not consider corresponding Y value

**Step6:** Use simple majority of nearest neighbors we have Two Goods and One Bad  
Therefore X1=3,X2=7 will goes into Good category

## **Choosing the right value for K:**

- To select the K that's right for your data, we run the KNN algorithm several times with different values of K and choose the K that reduces the number of errors we encounter while maintaining the algorithm's ability to accurately make predictions when it's given data it hasn't seen before.

### **Here are some things to keep in mind:**

- As we decrease the value of K to 1, our predictions become less stable. If  $K=1$  our query will deal with only one neighbor it leads to overfit the data.
- Inversely, as we increase the value of K, our predictions become more stable due to majority voting / averaging, and thus, more likely to make more accurate predictions (up to a certain point). Eventually, we begin to witness an increasing number of errors. It is at this point we know we have pushed the value of K too far.
- In cases where we are taking a majority vote (e.g. picking the mode in a classification problem) among labels, we usually make K an odd number to have a tiebreaker.

## **Advantages:**

- The algorithm is simple and easy to implement.
- There's no need to build a model, tune several parameters, or make additional assumptions.
- The algorithm is versatile. It can be used for classification, and regression.

## **Disadvantages:**

- The algorithm gets significantly slower as the number of examples and/or predictors/independent variables increase.

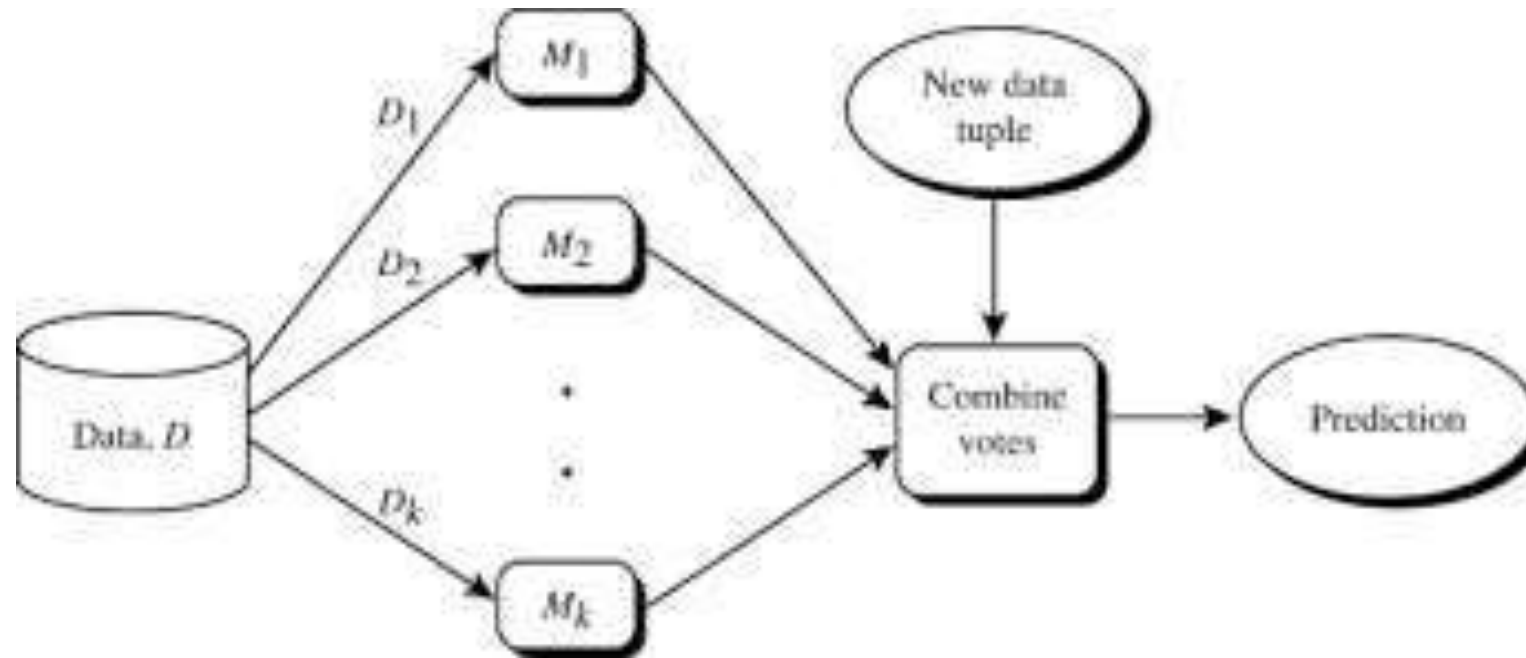
# Ensemble Methods

---



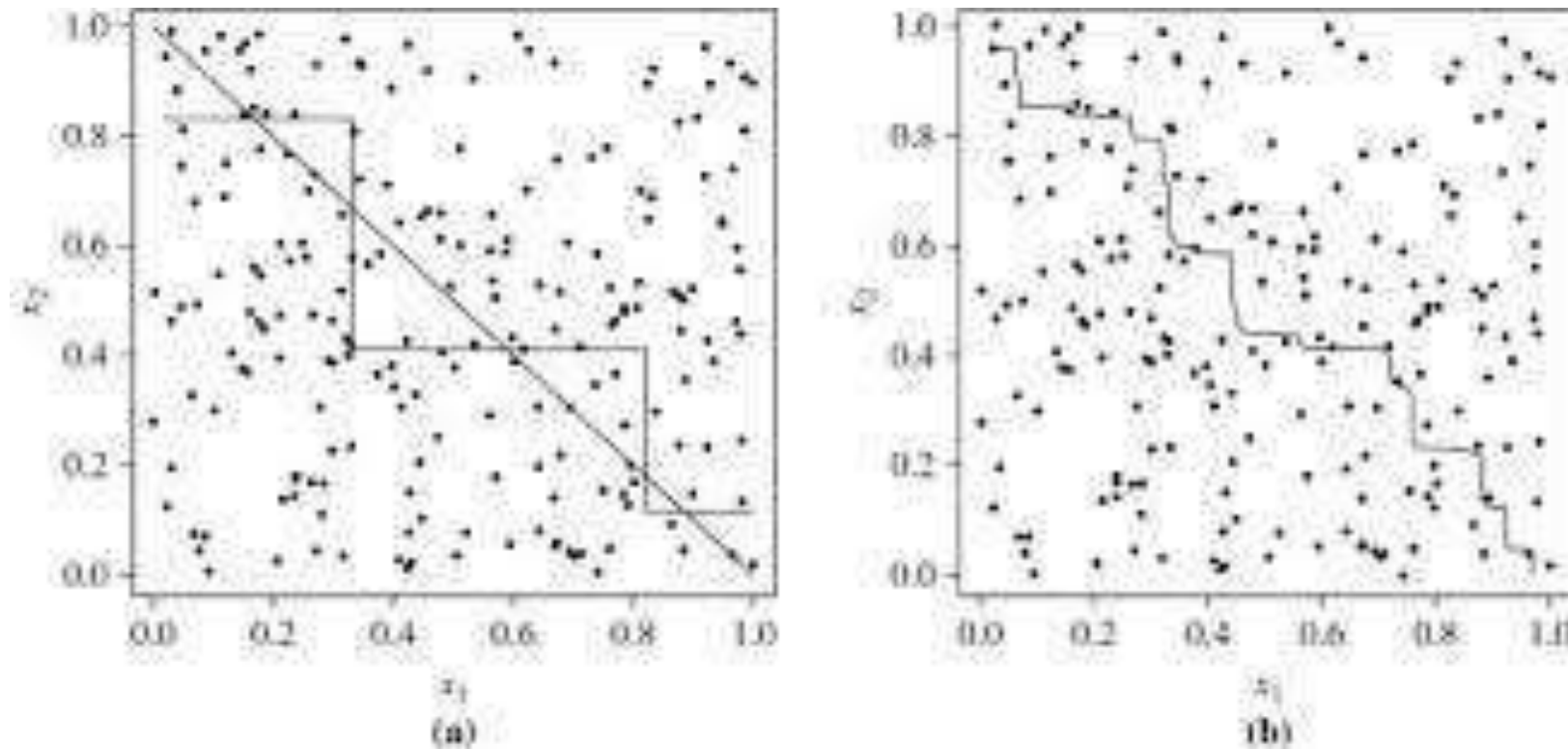
# Ensemble Methods:

- Ensemble Methods are used for increasing classification accuracy
- An ensemble for classification is a composite model, made up of a combination of classifiers.
- The individual classifiers vote, and a class label prediction is returned by the ensemble based on the collection of votes.
- Ensembles tend to be more accurate than their component classifiers.



*Bagging, boosting, and random forests* are examples of **ensemble methods**

- In figure An ensemble combines a series of  $k$  learned models (or *base classifiers*),  $M_1, M_2, \dots, M_k$ ,
  - with the aim of creating an improved composite classification model,  $M$ .
  - A given data set,  $D$ , is used to create  $k$  training sets,  $D_1, D_2, \dots, D_k$ ,  $D_i (1 \leq i \leq k-1)$  is used to generate classifier  $M_i$ .
  - Given a new data tuple to classify, the base classifiers each vote by returning a class prediction.
  - The ensemble returns a class prediction based on the votes of the base classifiers.
  - An ensemble tends to be more accurate than its base classifiers.
- 
- For example, consider an ensemble that performs majority voting.
  - That is, given a tuple  $X$  to classify, it collects the class label predictions returned from the base classifiers and outputs the class
  - in majority.
  - The base classifiers may make mistakes, but the ensemble will misclassify  $X$  only if over half of the base classifiers are in error. Ensembles yield better results when there is significant diversity among the models.
  - That is, ideally, there is little correlation among classifiers.
  - The classifiers should also perform better than random guessing. Each base classifier can be allocated to a different CPU and so ensemble methods are parallelizable.



Decision boundary by (a) a single decision tree and (b) an ensemble of decision trees for a linearly separable problem (i.e., where the actual decision boundary is a straight line). The decision tree struggles with approximating a linear boundary. The decision boundary of the ensemble is closer to the true boundary

## Bootstrap:

- The **bootstrap method** samples the given training tuples **uniformly *with replacement***.
- That is, each time a tuple is selected, it is equally likely to be selected again and re-added to the training set.
- For instance, imagine a machine that randomly selects tuples for our training set.
- In *sampling with replacement*, the machine is allowed to select the same tuple more than once.
- There are several bootstrap methods. A commonly used one is the **.632 bootstrap**, which works as follows.
- Suppose we are given a data set of  $d$  tuples. The data set is sampled  $d$  times, with replacement, resulting in a *bootstrap sample* or training set of  $d$  samples.
- It is very likely that some of the original data tuples will occur more than once in this sample.
- The data tuples that did not make it into the training set end up forming the test set.
- Suppose we were to try this out several times. As it turns out, on average, 63.2% of the original data tuples will end up in the bootstrap sample, and the remaining 36.8% will form the test set (hence, the name, .632 bootstrap).

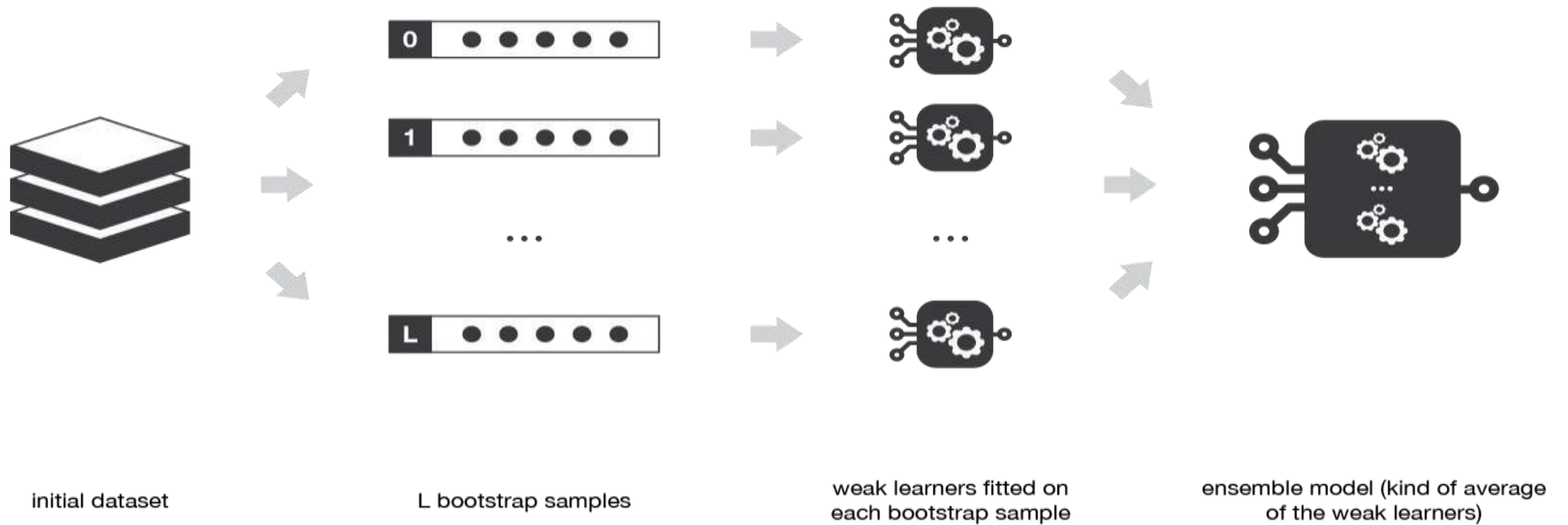


**“Where does the figure, 63.2%, come from?”**

- Each tuple has a probability of being selected  $= 1/d$
- so the probability of not being chosen is  $= (1-(1/d))$ .
- We have to select  $d$  times, so the probability that a tuple  
will not be chosen during this whole time is  $=(1-(1/d))^d$ .
- If  $d$  is large, the probability approaches  $e^{-1} = 0.368$ .
- Thus, 36.8% of tuples will not be selected for training and thereby end up in the test set, and the remaining 63.2% will form the training set.

## Bagging:

- We now take an intuitive look at how bagging works as a method of increasing accuracy.
  - Suppose that you are a patient and would like to have a diagnosis made based on your symptoms.
  - Instead of asking one doctor, you may choose to ask several.
  - If a certain diagnosis occurs more than any other, you may choose this as the final or best diagnosis.
  - That is, the final diagnosis is made based on a majority vote, where each doctor gets an equal vote.
  - Now replace each doctor by a classifier, and you have the basic idea behind bagging.
  - Intuitively, a majority vote made by a large group of doctors may be more reliable than a majority vote made by a small group.
- 
- Given a set,  $D$ , of  $d$  tuples, **bagging** works as follows.  
For iteration  $i$  ( $i = 1, 2, \dots, k$ ) a training set,  $D_i$ , of  $d$  tuples is **sampled with replacement** from the original set of tuples,  $D$ . Note that the term ***bagging* stands for *bootstrap aggregation***.  
Each training set is a bootstrap sample,
  - Because sampling with replacement is used, some of the original tuples of  $D$  may not be included in  $D_i$ , whereas others may occur more than once. A classifier model,  $M_i$ , is learned for each training set,  $D_i$ .
  - To classify an unknown tuple,  $\mathbf{X}$ , each classifier,  $M_i$ , returns its class prediction, which counts as one vote.
  - The bagged classifier,  $M$ , counts the votes and assigns the class with the most votes to  $\mathbf{X}$ .
  - Bagging can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple. The



Finally, we can mention that one of the big advantages of bagging is that **it can be parallelised**. As the different models are fitted independently from each others, intensive parallelisation techniques can be used if required.

## Boosting:

- The term 'Boosting' refers to a family of algorithms which converts weak learner to strong learners.
- Let's understand this definition in detail by solving a problem of spam email identification:
- How would you classify an email as SPAM or not? Like everyone else, our initial approach would be to identify 'spam' and 'not spam' emails using following criteria. If:
  - Email has only one image file (promotional image), It's a SPAM
  - Email has only link(s), It's a SPAM
  - Email body consist of sentence like "You won a prize money of \$ xxxxxx", It's a SPAM
  - Email from our official domain "[metu.edu.tr](http://metu.edu.tr)" , Not a SPAM
  - Email from known source, Not a SPAM
- Above, we've defined multiple rules to classify an email into 'spam' or 'not spam'. But, do you think these rules individually are strong enough to successfully classify an email? No.
- Individually, these rules are not powerful enough to classify an email into 'spam' or 'not spam'. Therefore, these rules are called as **weak learner**.

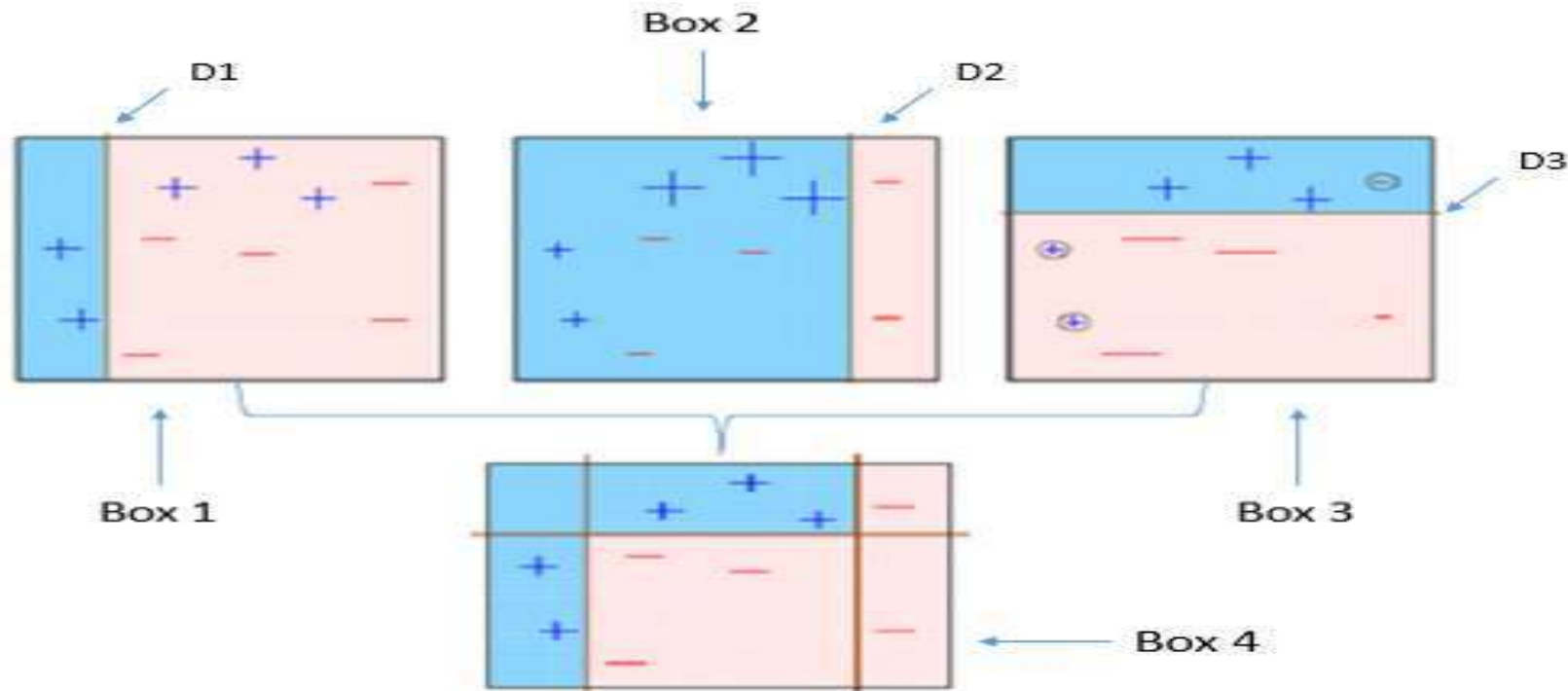


- To convert weak learner to strong learner, we'll combine the prediction of each weak learner using methods like:
  - Using average/ weighted average
  - Considering prediction has higher vote
- For example: Above, we have defined 5 weak learners. Out of these 5, 3 are voted as 'SPAM' and 2 are voted as 'Not a SPAM'. In this case, by default, we'll consider an email as SPAM because we have higher(3) vote for 'SPAM'.

### **How Boosting Algorithms works?**

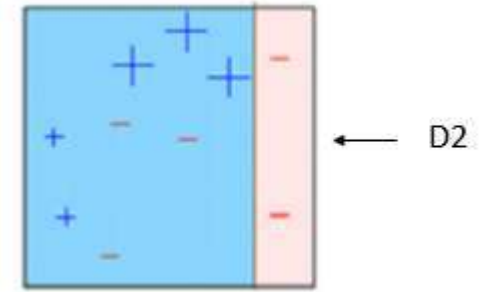
- To find weak rule, we apply base learning algorithms with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.
- For choosing the right distribution, here are the following steps:
  - Step 1:** The base learner takes all the distributions and assign equal weight or attention to each observation.
  - Step 2:** If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.
  - Step 3:** Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.
- Finally, it combines the outputs from weak learner and creates a strong learner which eventually improves the prediction power of the model. Boosting pays higher focus on examples which are misclassified or have higher errors by preceding weak rules.

# AdaBoost(Adaptive Boosting):

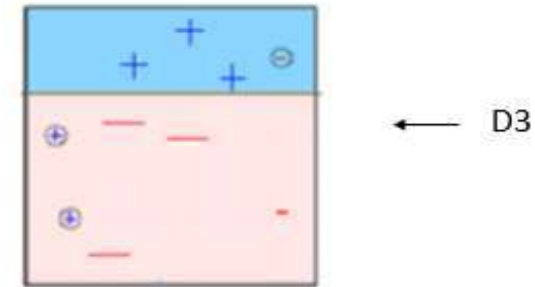


**Box 1:** You can see that we have assigned equal weights to each data point and applied a decision stump to classify them as + (plus) or – (minus). The decision stump (D1) has generated vertical line at left side to classify the data points. We see that, this vertical line has incorrectly predicted three + (plus) as – (minus). In such case, we'll assign higher weights to these three + (plus) and apply another decision stump.

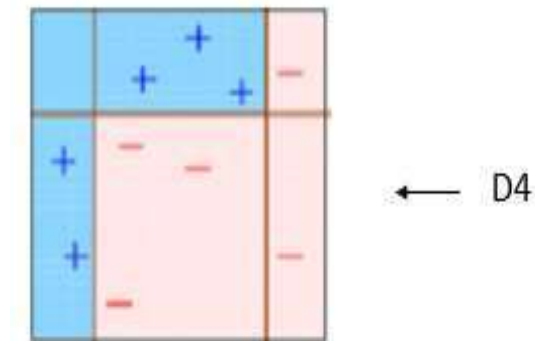
**Box 2:** Here, you can see that the size of three incorrectly predicted + (plus) is bigger as compared to rest of the data points. In this case, the second decision stump (D2) will try to predict them correctly. Now, a vertical line (D2) at right side of this box has classified three misclassified + (plus) correctly. But again, it has caused misclassification errors. This time with three -(minus). Again, we will assign higher weight to three - (minus) and apply another decision stump.



**Box 3:** Here, three - (minus) are given higher weights. A decision stump (D3) is applied to predict these misclassified observation correctly. This time a horizontal line is generated to classify + (plus) and - (minus) based on higher weight of misclassified observation.



**Box 4:** Here, we have combined D1, D2 and D3 to form a strong prediction having complex rule as compared to individual weak learner. You can see that this algorithm has classified these observation quite well as compared to any of individual weak learner.



## Math's Behind this Algo:

- How do we assign weight to observations?
- We always start with a uniform distribution assumption. Lets call it as  $D_1$  which is  $1/n$  for all  $n$  observations.
- Step 1 . We assume an  $\alpha(t)$
- Step 2: Get a weak classifier  $h(t)$
- Step 3: Update the population distribution for the next step

where

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

$$Z_t = \sum_{i=1}^m D_t(i) \exp(-\alpha_t y_i h_t(x_i))$$

Output the final hypothesis

$$H(x) = \text{sign} \left( \sum_{t=1}^T \alpha_t h_t(x) \right) .$$

# Gradient Boosting:

- In gradient boosting, it trains many models sequentially. Each new model gradually minimizes the loss function ( $y = ax + b + e$ ,  $e$  needs special attention as it is an error term) of the whole system using Gradient Descent method. The learning procedure consecutively fit new models to provide a more accurate estimate of the response variable.
- The principle idea behind this algorithm is to construct new base learners which can be maximally correlated with negative gradient of the loss function, associated with the whole ensemble.
- Assume, you are given a previous model  $M$  to improve on. Currently you observe that the model has an accuracy of 80% (any metric). How do you go further about it?
- One simple way is to build an entirely different model using new set of input variables and trying better ensemble learners. On the contrary, I have a much simpler way to suggest. It goes like this:

$$Y = M(x) + \text{error}$$

- What if I am able to see that error is not a white noise but have same correlation with outcome( $Y$ ) value. What if we can develop a model on this error term? Like,

$$\text{error} = G(x) + \text{error}_2$$

- Probably, you'll see error rate will improve to a higher number, say 84%. Let's take another step and regress against error2.

$$\text{error2} = H(x) + \text{error3}$$

- Now we combine all these together :

$$Y = M(x) + G(x) + H(x) + \text{error3}$$

- This probably will have an accuracy of even more than 84%. What if I can find an optimal weights for each of the three learners,

$$Y = \alpha * M(x) + \beta * G(x) + \gamma * H(x) + \text{error4}$$

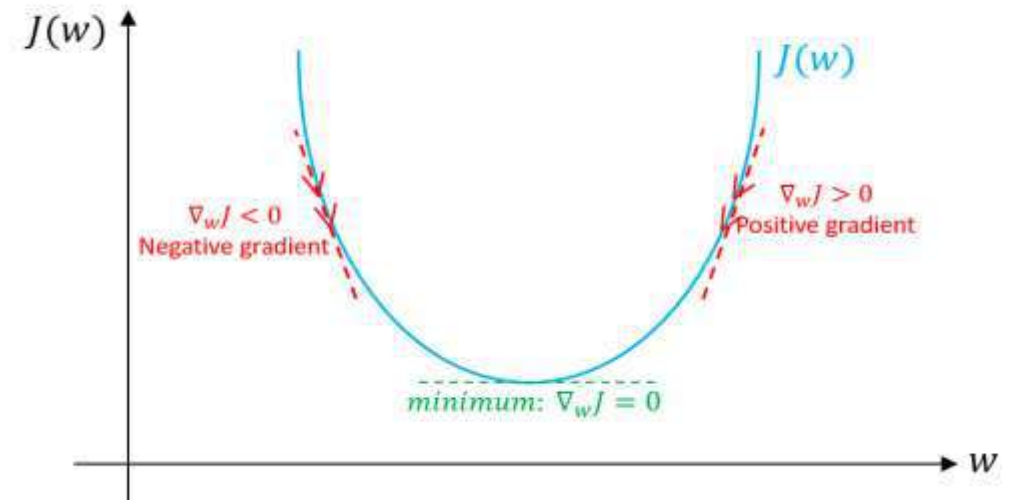
# Gradient Descent:

Gradient descent is an optimization algorithm used to minimize some function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient. In machine learning, we use gradient descent to update the parameters of our model. Parameters refer to coefficients in Linear Regression and weights in neural networks.

$$\theta_i := \theta_i - \rho \frac{\partial J}{\partial \theta_i}$$

## Learning rate( $\rho$ ):

The size of these steps is called the *learning rate*. With a high learning rate we can cover more ground each step, but we risk overshooting the lowest point since the slope of the hill is constantly changing. With a very low learning rate, we can confidently move in the direction of the negative gradient since we are recalculating it so frequently. A low learning rate is more precise, but calculating the gradient is time-consuming, so it will take us a very long time to get to the bottom.



# Bagging vs Boosting

- No clear winner; usually depends on the data
- Bagging is computationally more efficient than boosting (note that bagging can train the  $M$  models in parallel, boosting can't)
- Both reduce variance (and overfitting) by combining different models
- The resulting model has higher stability as compared to the individual ones
- Bagging usually can't reduce the bias, boosting can (note that in boosting, the training error steadily decreases)
- Bagging usually performs better than boosting if we don't have a high bias and only want to reduce variance (i.e., if we are overfitting)



# XGBoosting (Extreme Gradient Boosting)

- Ever since its introduction in 2014, XGBoost has been lauded as the holy grail of machine learning hackathons and competitions. From predicting ad click-through rates to classifying high energy physics events, XGBoost has proved its mettle in terms of performance – and speed.
- Execution Speed: Generally, XGBoost is fast. Really fast when compared to other implementations of gradient boosting. But newly introduced LightGBM is faster than XGBoosting.
- Model Performance: XGBoost dominates structured or tabular datasets on classification and regression predictive modeling problems. The evidence is that it is the go-to algorithm for competition winners on the Kaggle competitive data science platform
- The XGBoost library implements the gradient boosting decision tree algorithm.
- This algorithm goes by lots of different names such as gradient boosting, multiple additive regression trees, stochastic gradient boosting or gradient boosting machines.
- Boosting is an **ensemble technique** where new models are added to correct the errors made by existing models. Models are added sequentially until no further improvements can be made. A popular example is the AdaBoost algorithm that weights data points that are hard to predict.
- Gradient boosting is an approach where new models are created that predict the residuals or errors of prior models and then added together to make the final prediction. It is called gradient boosting because it uses a gradient descent algorithm to minimize the loss when adding new models.
- This approach supports both regression and classification predictive modeling problems.

- Boosting consists of three simple steps:
- An initial model  $F_0$  is defined to predict the target variable  $y$ . This model will be associated with a residual  $(y - F_0)$
- A new model  $h_1$  is fit to the residuals from the previous step
- Now,  $F_0$  and  $h_1$  are combined to give  $F_1$ , the boosted version of  $F_0$ . The mean squared error from  $F_1$  will be lower than that from  $F_0$ :

$$F_1(x) \leftarrow F_0(x) + h_1(x)$$

- To improve the performance of  $F_1$ , we could model after the residuals of  $F_1$  and create a new model  $F_2$ :

$$F_2(x) \leftarrow F_1(x) + h_2(x)$$

- This can be done for ' $m$ ' iterations, until residuals have been minimized as much as possible:

$$F_m(x) \leftarrow F_{m-1}(x) + h_m(x)$$

- Here, the additive learners do not disturb the functions created in the previous steps. Instead, they impart information of their own to bring down the errors.

# Unique features of XGBoost

- XGBoost is a popular implementation of gradient boosting. Let's discuss some features of XGBoost that make it so interesting.
- **Regularization:** XGBoost has an option to penalize complex models through both L1 and L2 regularization. Regularization helps in preventing overfitting
- **Handling sparse data:** Missing values or data processing steps like one-hot encoding make data sparse. XGBoost incorporates a sparsity-aware split finding algorithm to handle different types of sparsity patterns in the data

# **Support Vector Machines**

---

## Little Math's:

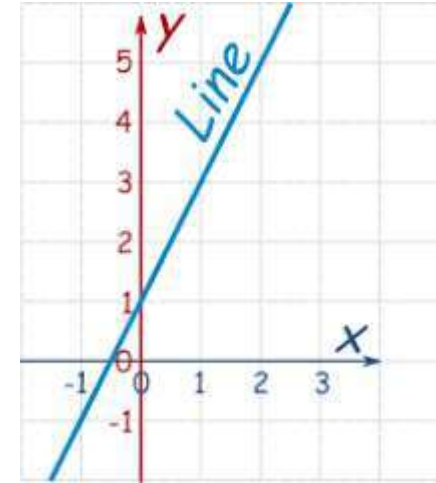
### Line:

When we talk about 2-Dimensional, let assume  $x, y$  forms 2D space

*Then the line equation is  $y = mx + c$  where  $m$ =slope,  $c$ =constant*

We can write another form of equation  $ax+by+c=0 \implies y = -\left(\frac{b}{a}\right)x - \left(\frac{c}{a}\right)$

Suppose if we take  $x_1, x_2$  are forms 2D space



# Thank You.

---

Omkar N